DGA Classifier from Domain Names: Family-First or Direct Binary?

Roudy Barakat rtb07@mail.aub.edu
Abdo Sebailli ams86@mail.aub.edu
American University of Beirut

1 Problem and Motivation

Malware uses Domain Generation Algorithms (DGAs) to generate many domains from a shared seed, so infected hosts can reach the attacker without a hard-coded address that defenders could block [12]. Real detection also uses network signals such as the NXDOMAIN rate and query periodicity [1]; this project focuses on detecting DGAs from the domain name alone. The difficulty is that "DGA" is not one pattern: some families produce random strings (`gqspwpsurdun.ru`, CryptoLocker) or hex hashes (`2c3f1e3326cca35b757ec704b2413a30.org`, Bamital), while others join dictionary words (`kneemanualgirl.com`, `whetherspread.net`) and look much closer to legitimate names.

Question. When the final decision is DGA vs. legitimate, should a classifier first learn the individual DGA families (each family + legitimate, with the prediction collapsed to DGA/legitimate) or learn DGA vs. legitimate directly?

Why it matters. The answer is for teams that build name-based DGA filters, for example in DNS resolvers or security products, where a model must judge a single query before any connection is made. If family-first training wins, family labels are worth collecting even when only a binary decision is needed, and simpler models may become accurate enough to deploy. If it does not, the simpler binary setup is enough.

Goal. For each model we evaluate, measure the difference in macro-F1 on the DGA/legitimate decision between family-first and direct binary training, on the same test set and averaged over 5 random seeds. We count family-first as better when its mean macro-F1 is higher by more than the standard deviation across seeds.

2 Impact

If it succeeds. Teams that build name-based DGA filters get a clear rule for how to train them: if family-first wins, it is worth collecting and maintaining family labels even when only a DGA/legitimate verdict is shipped; if both setups tie, the cheaper binary labels are enough. The per-family results also show which kinds of DGA (random-string or dictionary-based) a name-only detector can and cannot catch, which tells analysts where to expect blind spots.

If it fails. If neither setup separates some families from legitimate domains, in particular dictionary-based ones, that is evidence that the name alone does not carry enough signal for them. Detection would then need network traffic features, such as the NXDOMAIN rate, the periodicity of queries, and how many distinct, previously unseen domains a single host queries [1]. Knowing which families need these signals is itself a useful result.

Risks of using the result.

- *Errors do not cost the same.* A missed DGA domain (false negative) leaves an infected host talking to its attacker. A legitimate domain flagged as DGA (false positive) blocks a real service and adds to alert fatigue. Since almost all real DNS traffic is legitimate, even a small false-positive rate produces many false alarms.
- *Bias in the data.* Legitimate examples come from a popularity ranking (Tranco [8]), which over-represents popular sites. Legitimate but unusual names, such as non-English, short or machine-generated CDN hostnames, may be flagged unfairly. Families with fewer than 300 known domains are left out of our data, so our results say nothing about them.
- *Limited validity.* We test only families seen in training. Attackers can change their algorithms, so results do not guarantee detection of new or adversarial DGAs.

3 Related Work

Woodbridge et al. [16] introduced deep learning for DGA detection with a character-level LSTM trained on Alexa domains and 30 DGA families. They built a binary model and a separate family model: the binary model reached an AUC of 0.9993, but the family model's macro-F1 was only 0.54, rising to 0.96 when similar families were grouped. We adopt macro-F1, and their grouping result suggests that the labels matter. They never collapsed the family model back into a DGA/legitimate detector, which is the comparison we make.

Gregório et al. [6] classified 61 DGArchive families with a character-level CNN, reaching an average F1 of 84%, with some families at 0%. Their model has no legitimate class, so it cannot be used as a detector. We reuse this CNN, add the legitimate class, and test whether confusions between DGA families still matter once predictions are collapsed to DGA/legitimate.

Torrealba Aravena et al. [14] proposed DeepDGA, which combines a character-level BiLSTM with word embeddings for binary detection on 25 families. It exceeds 0.97 precision on dictionary-based DGAs, where feature-based models stay below 0.89. We take dictionary DGAs as the hardest case and report them separately. They close that gap with a bigger architecture; we test whether richer labels help simpler models.

Shahzad et al. [13] compared LSTM, BiLSTM and GRU for binary detection on over two million domains. The LSTM reached 87% accuracy and an AUC of about 0.98, with little difference between the three. We follow their preprocessing (removing duplicates after stripping the TLD) and their 80/10/10 train/validation/test split. Since changing the architecture barely mattered, we change the training labels instead, and report per-family results, which they do not.

Dahal and Kim [3] compressed character-count vectors with an autoencoder and classified them with a neural network or an SVM on 11 DGA families. They reached 88% accuracy, which fell to 78% on families not seen in training. We use character counts, as they do, as a simple input for our baseline. They target unseen families, while we focus on known ones; their drop shows the limit of our stratified split.

4 Data

Sources.

- *DGA:* Gregório and Cansian, Mendeley Data v1, 2025, CC BY 4.0, `https://data.mendele`

y.com/datasets/nhvyvytn2h/1 [5]: 3,090,661 domains in 160 families, from DGArchive [12] (Sep–Oct 2025).

- *DGA:* C. Givre, Kaggle "DGA dataset" v1, 2021, license not stated (academic use), `https://www.kaggle.com/datasets/gtkcyber/dga-dataset` [4]: 80,000 domains in 7 families.
- *Legitimate:* Tranco top 1M [8], list `64VVX` (17 Sep–16 Oct 2026), `https://tranco-list.eu/list/64VVX`. The legitimate rows of the two DGA datasets are not used.

Size, input and target. 1,999,335 domains: 999,664 legitimate and 999,671 DGA in 103 families (Fig. 1, Table 1). The input is the full domain name, TLD included (4–93 characters); all features are derived from it. The targets are the binary label and the family (`legit` or one of 103). We use pandas [9] and Matplotlib [7].

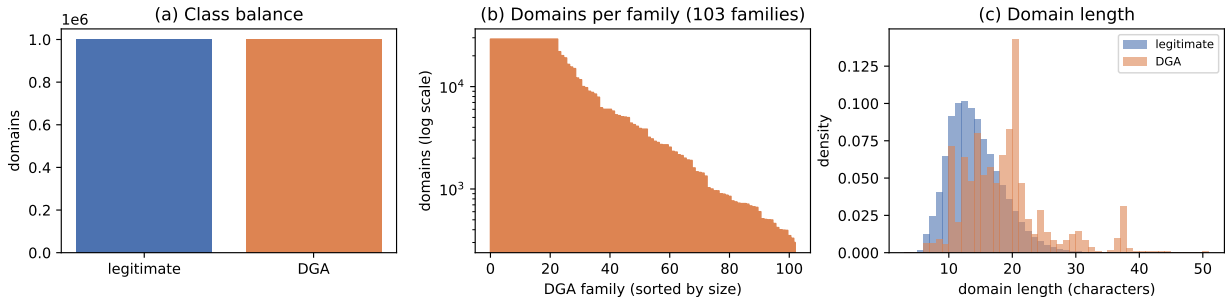


Figure 1: (a) Class sizes; (b) domains per DGA family (log scale); (c) domain length.

Table 1: Statistics per class. Entropy (bits) and digit ratio are computed on the part before the TLD.

Class	Domains	Families	Mean length	Entropy	Digit ratio	TLDs	Top TLD
Legitimate	999,664	–	13.7	2.79	0.038	1,030	.com (42%)
DGA	999,671	103	18.4	3.17	0.149	273	.com (23%)

Cleaning. There are no missing values. We drop 330 domains found in both Tranco and a DGA list (e.g. `pywolwnvd.biz`, family `m0yv`), 141 duplicates and 7 invalid strings. Kaggle family names are mapped to Mendeley names by domain shape (e.g. `newgoz` → `gameover`).

Balancing. After cleaning, the 161 DGA families range from 2 to 519,280 domains. We keep all legitimate domains, merge the 59 families with fewer than 300 domains into `other_dga`, and cap each family at 29,120 domains so that DGA and legitimate are equal in number. Sample weights give legitimate and DGA half each and, within DGA, are proportional to $1/\sqrt{n_f}$ (n_f: family size), so no family weighs more than 9.9 times another. The data are split 80/10/10 (train/validation/test), stratified by family.

Biases. Tranco favours popular sites. DGA families use few TLDs (23 use only one, vs. 1,030 TLDs for legitimate domains), so the TLD alone separates some families. The DGA data cover only 2021 and Sep–Oct 2025.

5 Proposed Models and Methods

Every model is trained in two setups on the same training set: *binary* (target DGA/legitimate) and *family-first* (target `legit` or one of the 97 families, collapsed with $P(\text{DGA}) = 1 - P(\texttt{legit})$). The neural models are also trained *multi-task* [2]: one shared encoder with a binary head and a

family head whose losses are summed, so the family labels shape the latent space while the binary head makes the decision. Hyperparameters are chosen on the validation set.

Model 0: majority class. Always predicts the most frequent class. It is the floor every model must beat.

Model 1: logistic regression (scikit-learn [11]). *Why it suits the data:* it is fast and interpretable, and it is linear: a binary model draws a single boundary between legitimate names and all DGAs, although families sit in very different regions (random strings vs. dictionary words). *What we learn:* family-first gives it one boundary per family, so if family labels help anywhere, they should help most here.

Model 2: character-level CNN [6], in PyTorch [10]. It embeds each character and applies convolutions, learning local character patterns (hex runs, alternating vowels and consonants, word pieces) wherever they occur in the name. *Why it suits the data:* families differ mostly in such local patterns. *What we learn:* comparing it with Model 1 shows whether the gain from family labels shrinks as the model becomes more flexible, and the multi-task setup shows whether that gain can be kept without the collapse step.

Model 3: character-level Transformer encoder [15]. Self-attention relates characters anywhere in the name, not only neighbours. *Why it suits the data:* dictionary DGAs, the hardest case [14], are built from whole words, so their pattern spans the name. *What we learn:* whether a model that sees the whole name at once closes the gap on dictionary families, and how the three setups compare for it. Finally, we test whether the CNN or Transformer embeddings, given to the simple Model 1, are enough to match the full neural models.

References

[1] Manos Antonakakis, Roberto Perdisci, Yacin Nadji, Nikolaos Vasiloglou, Saeed Abu-Nimeh, Wenke Lee, and David Dagon. From throw-away traffic to bots: Detecting the rise of DGA-based malware. In *Proceedings of the 21st USENIX Security Symposium*, pages 491–506, 2012.

[2] Rich Caruana. Multitask learning. *Machine Learning*, 28(1):41–75, 1997. doi: 10.1023/A:1007379606734.

[3] Binay Dahal and Yoohwan Kim. Autoencoded domains with mean activation for DGA botnet detection. In *Proceedings of the 12th IEEE International Conference on Global Security, Safety and Sustainability (ICGS3)*. IEEE, 2019.

[4] Charles Givre. DGA dataset. `https://www.kaggle.com/datasets/gtkcyber/dga-dataset`, 2021. Version 1; accessed October 2026.

[5] João Rafael Gregório and Adriano Mauro Cansian. Legitimate domains and DGA categorized morphologically and by families, 2025. URL `https://data.mendeley.com/datasets/nhvyvytn2h/1`.

[6] João Rafael Gregório, Adriano Mauro Cansian, and Leandro Alves Neves. Class incremental deep learning: A computational scheme to avoid catastrophic forgetting in domain generation algorithm multiclass classification. *Applied Sciences*, 14(16):7244, 2024. doi: 10.3390/app14167244.

[7] John D. Hunter. Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007. doi: 10.1109/MCSE.2007.55.

[8] Victor Le Pochat, Tom Van Goethem, Samaneh Tajalizadehkhoob, Maciej Korczyński, and Wouter Joosen. Tranco: A research-oriented top sites ranking hardened against manipulation. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. Internet Society, 2019. doi: 10.14722/ndss.2019.23386.

[9] Wes McKinney. Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference*, pages 56–61, 2010. doi: 10.25080/Majora-92bf1922-00a.

[10] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, pages 8024–8035, 2019.

[11] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[12] Daniel Plohmann, Khaled Yakdan, Michael Klatt, Johannes Bader, and Elmar Gerhards-Padilla. A comprehensive measurement study of domain generating malware. In *Proceedings of the 25th USENIX Security Symposium*, pages 263–278, 2016.

[13] Haleh Shahzad, Abdul Rahman Sattar, and Janahan Skandaraniyam. DGA domain detection using deep learning. In *Proceedings of the 5th IEEE International Conference on Cryptography, Security and Privacy (CSP)*. IEEE, 2021. doi: 10.1109/CSP51677.2021.9357591.

[14] Lucas Torrealba Aravena, Pedro Casas, Diego García, Javier Bustos-Jiménez, and Ivana Bachmann. Malicious domain names detection with DeepDGA, a hybrid character and word embeddings deep learning architecture. In *Proceedings of the 21st International Conference on Network and Service Management (CNSM)*. IFIP, 2025. doi: 10.23919/CNSM67658.2025.11297537.

[15] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 5998–6008, 2017.

[16] Jonathan Woodbridge, Hyrum S. Anderson, Anjum Ahuja, and Daniel Grant. Predicting domain generation algorithms with long short-term memory networks, 2016.